

Методы списков и строк

В материалах занятия приводятся таблицы с почти полным перечнем методов списков и строк, которые можно использовать как справочный материал. Рассматривается неявное приведение объектов к булеву типу. Приводятся примеры цепочек вызова методов. Появление метода *pop* позволяет познакомиться с понятием стека.

Это занятие отличается от прочих: большую часть его материала не нужно запоминать, но можно использовать как справочный материал. С этой же целью вводятся функции *dir* и *help*.

Как пользоваться таблицами

Мы уже знакомы с некоторыми функциями для работы со списками, строками и множествами. Мы знаем также методы для работы с элементами множеств (*add*, *remove*, *discard* и т. д.), метод *append*, который позволяет добавлять элементы в список, метод *split* для разбиения строки на список «слов» и метод *join* для «сшивки» списка строк в одну. Однако мы не знаем, как делать со списками многие другие вещи: добавлять элементы в произвольное место, удалять элементы и т. д. Такие задачи решаются с помощью других функций и методов.

Далее приводится перечень основных методов списков и строк, а также функций и специальных синтаксических конструкций для работы с ними, включая уже знакомые нам. Если при написании программы вы забудете, как называется тот или иной метод или какие у него аргументы, смело заглядывайте в материалы этого урока, в [документацию](#), пользуйтесь поиском в Интернете или описанными здесь функциями *dir* и *help*.

Важно!

Главное — усвоить основные возможности методов для стандартных типов строки и списка, а детали (например, порядок аргументов в методе *insert* или название именованного аргумента в методе *split*) всегда можно посмотреть здесь или в документации.

В таблице предполагается, что *a* и *a2* — списки, *x* — элемент списка, *s* и *s2* — строки, *c* — символ (строка длины 1), *n* — индекс элемента списка или строки, *start*, *stop*, *step* — границы среза (т. е. тоже индексы) и шаг среза, *k* — натуральное число.

Для методов и функций также даны примеры выражений, которые будут истинными True и демонстрируют действие этого метода. Например, выражения `5 in [2, 3, 5]` и `'abc'.isalpha()` равны True. Их можно подставить в оператор *if* (и соответствующий блок будет выполнен) или в функцию *print* (и тогда она выведет True).

Методы списков

Операция	Описание	Пример
<code>x in a</code>	Проверка, что <i>x</i> содержится в <i>a</i>	<code>5 in [2, 3, 5]</code>
<code>x not in a</code>	Проверка, что <i>x</i> не	<code>5 not in [2, 3, 6]</code>

	содержится в <i>a</i> То же, что и <code>not (x in a)</code>	
<code>a + a2</code>	Конкатенация списков, то есть новый список, в котором сначала идут все элементы <i>a</i> , а затем все элементы <i>a2</i>	<code>[2, 4] + [5, 3] == [2, 4, 5, 3]</code>
<code>a * k</code>	Список <i>a</i> , повторенный <i>k</i> раз	<code>[2, 3] * 3 == [2, 3, 2, 3, 2, 3]</code>
<code>a[n]</code>	<i>n</i> -й элемент списка, отрицательные <i>n</i> — для отсчета с конца	<code>[2, 3, 7][0] == 2</code> <code>[2, 3, 7][-1] == 7</code>
<code>a[start:stop:step]</code>	Срез списка	<code>[2, 3, 7][:2] == [2, 3]</code>
<code>len(a)</code>	Длина списка	<code>len([2, 3, 7]) == 3</code>
<code>max(a)</code>	Максимальный элемент списка	<code>max([2, 3, 7]) == 7</code>
<code>min(a)</code>	Минимальный элемент списка	<code>min([2, 3, 7]) == 2</code>
<code>sum(a)</code>	Сумма элементов списка	<code>sum([2, 3, 7]) == 12</code>
<code>a.index(x)</code>	Индекс первого вхождения <i>x</i> в <i>a</i> (вызовет ошибку, если <code>x not in a</code> , то есть если <i>x</i> отсутствует в <i>a</i>)	<code>[2, 3, 7].index(7) == 2</code>
<code>a.count(x)</code>	Количество вхождений <i>x</i> в <i>a</i>	<code>[2, 7, 3, 7].count(7) == 2</code>
<code>a.append(x)</code>	Добавить <i>x</i> в конец <i>a</i>	<code>a = [2, 3, 7]</code> <code>a.append(8)</code> <code>a == [2, 3, 7, 8]</code>
<code>a.extend(a2)</code>	Добавить элементы коллекции <i>a2</i> в конец <i>a</i>	<code>a = [2, 3, 7]</code> <code>a.extend([8, 4, 5])</code> <code>a == [2, 3, 7, 8, 4, 5]</code>
<code>del a[n]</code>	Удалить <i>n</i> -й элемент списка	<code>a = [2, 3, 7]</code> <code>del a[1]</code>

		<code>a == [2, 7]</code>
<code>del a[start:stop:step]</code>	Удалить из <code>a</code> все элементы, попавшие в срез	<code>a = [2, 3, 7]</code> <code>del a[:2]</code> <code>a == [7]</code>
<code>a.clear()</code>	Удалить из <code>a</code> все элементы (то же, что <code>del a[:]</code>)	<code>a.clear()</code>
<code>a.copy()</code>	Копия <code>a</code> (то же, что и полный срез <code>a[:]</code>)	<code>b = a.copy()</code>
<code>a += a2</code> <code>a *= k</code>	Заменить содержимое списка на <code>a + a2</code> и <code>a * k</code> соответственно	
<code>a.insert(n, x)</code>	Вставить <code>x</code> в <code>a</code> на позицию <code>n</code> , подвинув последующую часть дальше	<code>a = [2, 3, 7]</code> <code>a.insert(0, 8)</code> <code>a == [8, 2, 3, 7]</code>
<code>a.pop(n)</code>	Получить <code>n</code> -й элемент списка и одновременно удалить его из списка. Вызов метода без аргументов равносильен удалению последнего элемента: <code>a.pop() == a.pop(-1)</code>	<code>a = [2, 3, 7]</code> <code>a.pop(1) == 3</code> <code>a == [2, 7]</code>
<code>a.remove(x)</code>	Удалить первое вхождение <code>x</code> в <code>a</code> , в случае <code>x not in a</code> — ошибка	<code>a = [2, 3, 7]</code> <code>a.remove(3)</code> <code>a == [2, 7]</code>
<code>a.reverse()</code>	Изменить порядок элементов в <code>a</code> на обратный (перевернуть список)	<code>a = [2, 3, 7]</code> <code>a.reverse()</code> <code>a == [7, 3, 2]</code>
<code>a.sort()</code>	Отсортировать список по возрастанию	<code>a = [3, 2, 7]</code> <code>a.sort()</code> <code>a == [2, 3, 7]</code>
<code>a.sort(reverse=True)</code>	Отсортировать список по убыванию	<code>a = [3, 2, 7]</code> <code>a.sort(reverse = True)</code> <code>a == [7, 3, 2]</code>
<code>bool(a)</code>	Один из способов проверить список на пустоту	

	(возвращает True, если список непустой, и False в противном случае)	
--	---	--

Методы строк

Операция	Описание	Пример
<code>s2 in s</code>	Проверка, что подстрока <code>s2</code> содержится в <code>s</code>	<code>'m' in 'team'</code>
<code>s2 not in s</code>	Проверка, что подстрока <code>s2</code> не содержится в <code>s</code> то же, что <code>not (s2 in s)</code>	<code>'I' not in 'team'</code>
<code>s + s2</code>	Конкатенация (склейка) строк, то есть строка, в которой сначала идут все символы из <code>s</code> , а затем все символы из <code>s2</code>	<code>'tea' + 'm' == 'team'</code>
<code>s * k</code>	Строка <code>s</code> , повторенная <code>k</code> раз	<code>'ha' * 3 == 'hahaha'</code>
<code>s[n]</code>	<code>n</code> -й элемент строки, отрицательные <code>n</code> — для отсчета с конца	<code>'team'[2] == 'a'</code> <code>'team'[-1] == 'm'</code>
<code>s[start:stop:step]</code>	Срез строки	<code>'mama'[:2] == 'ma'</code>
<code>len(s)</code>	Длина строки	<code>len('abracadabra') == 11</code>
<code>s.find(s2)</code> <code>s.rfind(s2)</code>	Индекс начала первого или последнего вхождения подстроки <code>s2</code> в <code>s</code> (вернет -1, если <code>s2 not in s</code>)	<code>s = 'abracadabra'</code> <code>s.find('ab') == 0</code> <code>s.rfind('ab') == 7</code> <code>s.find('x') == -1</code>
<code>s.count(s2)</code>	Количество неперекрывающихся вхождений <code>s2</code> в <code>s</code>	<code>'abracadabra'.count('a') == 5</code>
<code>s.startswith(s2)</code> <code>s.endswith(s2)</code>	Проверка, что <code>s</code> начинается с <code>s2</code> или оканчивается на <code>s2</code>	<code>'abracadabra'.startswith('abra')</code>

<pre>s += s2 s *= k</pre>	Заменить содержимое строки на $s + s2$ и $s * k$ соответственно	
<pre>s.isdigit() s.isalpha() s.isalnum()</pre>	Проверка, что в строке s все символы — цифры, буквы (включая кириллические), цифры или буквы соответственно	<pre>'100'.isdigit() 'abc'.isalpha() 'E315'.isalnum()</pre>
<pre>s.islower() s.isupper()</pre>	Проверка, что в строке s не встречаются большие буквы, маленькие буквы. Обратите внимание, что для обеих этих функций знаки препинания и цифры дают True	<pre>'hello!'.islower() '123PYTHON'.isupper()</pre>
<pre>s.lower() s.upper()</pre>	Строка s, в которой все буквы (включая кириллические) приведены к верхнему или нижнему регистру, т. е. заменены на строчные (маленькие) или заглавные (большие)	<pre>'Привет!'.lower() == 'привет!' 'Привет!'.upper() == 'ПРИВЕТ!'</pre>
<pre>s.capitalize()</pre>	Строка s, в которой первая буква — заглавная	<pre>'привет'.capitalize() == 'Привет'</pre>
<pre>s.lstrip() s.rstrip() s.strip()</pre>	Строка s, у которой удалены символы пустого пространства (пробелы, табуляции) в начале, в конце или с обеих сторон	<pre>'Привет! '.strip() == 'Привет!'</pre>
<pre>s.ljust(k, c) s.rjust(k, c)</pre>	Добавляет справа или слева нужное количество символов c, чтобы	<pre>'Привет'.ljust(8, '!') == 'Привет!!!'</pre>

	длина s достигла k	
<code>s.join(a)</code>	Склеивает строки из списка a через символ s	<code>'+'.join(['Вася', 'Маша']) == 'Вася+Маша'</code>
<code>s.split(s2)</code>	Список всех слов строки s (подстрок, разделенных строками s2)	<code>'Раз два три!'.split('a') == ['Р', 'з дв', ' три!']</code>
<code>s.replace(s2, s3)</code>	Строка s, в которой все неперекрывающиеся вхождения s2 заменены на s3 Есть необязательный третий параметр, с помощью которого можно указать, сколько раз производить замену	<code>'Раз два три!'.replace('а', 'я') == 'Ряз двя три!'</code> <code>'Раз два три!'.replace('а', 'я', 1) == 'Ряз два три!'</code>
<code>list(s)</code>	Список символов из строки s	<code>list('Привет') == ['П', 'р', 'и', 'в', 'е', 'т']</code>
<code>bool(s)</code>	Проверка, что строка не пустая (возвращает True, если не пустая, и False в противном случае)	
<code>int(s)</code> <code>float(s)</code>	Если в строке s записано целое (дробное) число, получить это число, иначе — ошибка	<code>int('25') == 25</code>
<code>str(x)</code>	Представить любой объект x в виде строки	<code>str(25) == '25'</code>

Важно!

Обратите внимание: **никакие** методы строк, включая `s.replace(...)`, не изменяют саму строку `s`. Все они лишь **возвращают измененную строку**, в отличие от большинства методов списков.

`a.sort()`, например, ничего не возвращает, а изменяет сам список `a`.

Функции `dir` и `help`

Для получения информации о списке методов любого объекта (в том числе тех, о которых вы вряд ли хотели узнать) в Python существует специальная функция **`dir`**. Например, `dir([])` вернет все методы списков, и оператор `print(dir([]))` выведет длинный список, оканчивающийся так: `'index', 'insert', 'pop', 'remove', 'reverse', 'sort'`.

Если же нам нужно узнать справочную информацию по конкретному методу или типу данных, для этого существует функция **`help`**. `help([])` выведет на экран много информации, большая часть которой пока лишняя. А вот `help([].insert)` выведет на экран краткую справку именно по методу *`insert`*, подсказывая, что первый аргумент этого метода — индекс, а второй — тот объект, который нужно вставить в список на этот индекс.

Важно!

Заметьте: при вызове справки по методу `help([].insert)` после *`insert`* нет скобок (...) — ведь мы не хотим вызвать этот метод, чтобы вставить что-то в какой-то список. Функции *`help`* в качестве аргумента передается сам метод *`insert`*.

Форматированный вывод. f-строки

Этот блок урока не имеет отношения к методам списков и строк, но позволит нам познакомиться с возможностями форматированного вывода на экран переменных и выражений.

Начиная с версии 3.6 в Python появился новый тип строк — f-строки, которые улучшают читаемость кода и работают быстрее других способов форматирования.

f-строки

f-строки, которые буквально означают *formatted string*, задаются с помощью литерала `f` перед кавычками:

```
>>> "обычная строка"
>>> f"f-строка"
```

f-строки делают очень простую вещь: они берут значения переменных, которые есть в текущей области видимости, и подставляют их в строку. В самой строке вам лишь нужно указать имя этой переменной в фигурных скобках.

```
name = "Аркадий"
age = 14
print(f"Меня зовут {name} Мне {age} лет.")
```

Меня зовут Аркадий. Мне 14 лет.

В качестве подставляемого значения внутри фигурных скобок может быть: имя переменной (`f"Меня зовут {name}"`), элемент списка (`f"Третий месяц в году - {month[2]}"`) или словаря, методы объектов (`f"Имя: {name.upper()}"`). f-строки позволяют выполнять базовые арифметические операции (`f"({x} + {y}) / 2 = {(x + y) / 2}"`) и даже вызывать функции (`f"13 / 7 = {round(13/7)}"`).

Кроме того, вы можете задавать форматирование для чисел.

- Указать необходимое количество знаков после запятой, спецификатор *f* отвечает за вывод чисел с плавающей точкой (тип *float*):

```
print(f"Число пи по Архимеду - {(22/7):.2f}")
```

- Представить результат в двоичной системе счисления, используя спецификатор *b*:

```
print(f"10 в двоичной системе счисления - {(10):b}")
```

Аналогично для шестнадцатеричной системы счисления используется спецификатор *x*, а для восьмеричной — *o*.

Подробнее о допустимых вариантах форматирования можно почитать в [документации](#) или на вот этом [ресурсе](#).

Цепочки вызовов

Бывает удобно строить последовательные цепочки вызовов методов.

Например, `s.strip().lower().replace('ё', 'е')` выдаст строку *S*, в которой убраны начальные и конечные пробелы, все буквы сделаны маленькими, после чего убраны все точки над Ё. В результате этого преобразования строка ' Зелёный клён ' превратится в 'зеленый клен'.

Другой пример:

```
'мало Средне МНОГО'.lower().split().index('средне')
```

1

В данном примере строка сначала полностью приводится к нижнему регистру (все буквы маленькие) при помощи метода `lower()`. Затем она превращается в список слов ['мало', 'средне', 'много'] при помощи `split()`. Далее метод *index* находит в этом списке слово «средне» на позиции номер 1.

Использование методов списков. Структура данных «Стек»

Приведем пример использования методов списков *append* и *pop* для моделирования структуры данных «Стек». Эта структура данных часто используется при решении различных задач (например, при вычислении выражений).

Представьте себе стопку сложенных футболок или любых других неодинаковых предметов, которые можно сложить в стопку. Из стопки удобно забрать самый верхний предмет — тот, который положили в нее последним. Если вы полностью разберете стопку, будете брать футболки в порядке, обратном тому, в котором их в нее клали.

Стек

Структура данных называется стек (stack, что и значит «стопка»). По-английски такую организацию данных называют **LIFO — Last In First Out — Первым Пришел Последним Ушел!**

Если пользоваться только методами *append* и *pop* без аргументов, список оказывается как раз такой стопкой.

Так, в примере ниже порядок вывода будет обратным порядку ввода:

```
stack = []

for i in range(5):

    print('Какую футболку вы кладёте сверху стопки?')

    stack.append(input())

while stack: # пока стопка не закончилась

    print('Сегодня вы надеваете футболку',
stack.pop())
```

Какую футболку вы кладёте сверху стопки?

белую

Какую футболку вы кладёте сверху стопки?

красную

Какую футболку вы кладёте сверху стопки?

синюю

Какую футболку вы кладёте сверху стопки?

зеленую

Какую футболку вы кладёте сверху стопки?

веселую

Сегодня вы надеваете футболку веселую

Сегодня вы надеваете футболку зеленую

Сегодня вы надеваете футболку синюю

Сегодня вы надеваете футболку красную

Сегодня вы надеваете футболку белую

Здесь использовалась конструкция `while stack:`. В условии оператора *if* или *while* любой объект интерпретируется как *bool*: либо как `True`, либо как

False. В случае списков и строк в False превращаются только [] и '' соответственно (а в случае чисел — только ноль).

Иными словами, можно было бы написать `while bool(stack):` или `while stack != []:` и получить тот же самый результат, но самый короткий и общепринятый вариант — просто `while stack:`.

Комментарии в программе

Ограничение времени	1 секунда
Ограничение памяти	64Mb
Ввод	стандартный ввод или input.txt
Вывод	стандартный вывод или output.txt

Напишите программу, которая удаляет из программы на языке Python комментарии, а также символы пустого пространства на конце строк. Считайте, что символ решётки всегда означает начало комментария (проверьте себя: в каком случае это не так?).

Формат ввода

На первой строке вводится символ решётки и сразу же натуральное число N — количество строк в программе, не считая эту первую. Далее следует N строк.

Формат вывода

Нужно вывести те же строки, но удалить комментарии и символы пустого пространства в конце строк. Пустую строку вместо первой строки ввода выводить не надо.

Пример

Ввод

```
#2
name = input()
print('Привет,', name) # здороваемся
```

Вывод

```
name = input()
print('Привет,', name)
```

```
1 x = input()
2 x = x[1:]
3 a = []
4 for i in range(int(x)):
5     s = input()
6     s.split()
7     if '#' in s:
8         f = s.index('#')
9         s = ''.join(s[:f])
10    s = s.rstrip()
11    a.append(s)
12 for i in range(len(a)):
13     print(a[i], sep=" ")
14
```

Пример 1. Польский калькулятор

Ограничение времени	1 секунда
Ограничение памяти	64Mb
Ввод	стандартный ввод или input.txt
Вывод	стандартный вывод или output.txt

Напишите программу, которая производит вычисление выражения, записанного в обратной польской нотации (ОПН).

В ОПН нет ни скобок, ни приоритета операторов («умножение раньше сложения»). Чтобы прочитать выражение, записанное в ОПН, нужно просматривать выражение строго последовательно. Вводимые значения последовательно добавляются в стек, а когда встречается символ операции, то из стека извлекаются последние положенные туда значения, с ними производится эта операция, и результат возвращается в стек. Если для операции важен порядок значений, с которыми она производится, то первым идёт число, лежавшее в стеке глубже; в частности, если операция — вычитание, то из предпоследнего числа в стеке вычитается последнее, а не наоборот. Изначально стек пустой, в результате полного вычисления выражения в нём должно остаться одно значение — результат вычислений.

Первый пример следует читать так: в стек последовательно добавляются значения 7, 2, 3, затем встречаем знак операции *, поэтому значения 2 и 3 извлекаются, перемножаются, результат (6) кладётся обратно в стек, и следующий знак - извлекает из стека два оставшихся в нём значения 7 и 6, вычитает одно из другого и кладёт результат снова в стек. Выражение закончилось, в стеке одно число 1, это и есть результат вычисления.

Формат ввода

Вводится одна строка, содержащая разделённые пробелами целые числа и знаки операций +, -, *, вместе составляющие корректное выражение в обратной польской нотации.

Формат вывода

Выводится одно целое число — результат вычисления выражения.

Пример 1

Ввод

7 2 3 * -

Вывод

1

Пример 2

Ввод

10 15 - 7 *

Вывод

-35

```
1 s = ()
2 s = input()
3 s = s.split()
4 for x in s:
5     if x == '+':
6         i = s.index(x)
7         r = int(s[i - 2]) + int(s[i - 1])
8         s = s[:i - 2] + [r] + s[(i + 1):]
9     if x == '-':
10        i = s.index(x)
11        r = int(s[i - 2]) - int(s[i - 1])
12        s = s[:i - 2] + [r] + s[(i + 1):]
13    if x == '*':
14        i = s.index(x)
15        r = int(s[i - 2]) * int(s[i - 1])
16        s = s[:i - 2] + [r] + s[(i + 1):]
17    if x == '/':
18        i = s.index(x)
19        r = int(s[i - 2]) / int(s[i - 1])
20        s = s[:i - 2] + [r] + s[(i + 1):]
21 for i in range(len(s)):
22     print(s[i])
```

Пример 2. Знаков без пробелов

Ограничение времени

1 секунда

Ограничение памяти

64Mb

Ввод

стандартный ввод или input.txt

Вывод

стандартный вывод или output.txt

Часто бывает нужно сосчитать, сколько в тексте печатаемых знаков. Напишите программу, которая это делает.

Формат ввода

Вводится одна строка.

Формат вывода

Выводится одно целое число — количество символов в этой строке, не являющихся ни пробелами, ни табуляциями.

Пример

Ввод

Вывод

Раз два три

9

```
1 s = input()
2 k = 0
3 for x in s:
4     if x != ' ' and x != '\t':
5         k = k + 1
6 print(k)
```

Пример 3. Маленький частотный анализ

Ограничение времени

1 секунда

Ограничение памяти

64Mb

Ввод

стандартный ввод или input.txt

Вывод

стандартный вывод или output.txt

Частотный анализ — это подсчёт, какие символы чаще встречаются в тексте. Это важнейший инструмент взлома многих классических шифров — от шифра Цезаря до шифровальной машины «Энигма». Выполним простой частотный анализ: выясним, какой символ чаще всего встречается в данном тексте.

Формат ввода

Вводится одна строка.

Формат вывода

Выводится один символ в нижнем регистре — наиболее часто встречающийся во введённой строке, кроме пробела, без учёта регистра, если таких несколько — выводится первый по алфавиту.

Пример

Ввод

Вывод

Баобаb

b

```

1 string = ''.join(input().lower().split())
2 s = list()
3 big = 0
4 for x in string:
5     s.append(x)
6 s.sort()
7 for y in s:
8     if s.count(y) > big:
9         big = s.count(y)
10        let = y
11 print(let)

```

Пример 4. Пристраиваемся в очередь

Ограничение времени 1 секунда
 Ограничение памяти 64Mb
 Ввод стандартный ввод или input.txt
 Вывод стандартный вывод или output.txt

Ещё одна не очень достоверная модель очереди.

Изначально в очереди никого нет. Далее в каком-то порядке происходят события трёх типов:

- фраза «<фамилия> встал(а) в очередь.» означает, что пришёл новый человек и встал в конец очереди;
- фраза «Привет, <фамилия1>! <фамилия2> будет за тобой.» означает, что пришёл новый человек и встал в середину очереди за другим человеком, который там уже стоял;
- фраза «<фамилия>, хватит тут стоять, пошли отсюда.» означает, что кто-то покинул очередь.

Нужно определить, кто остался в очереди в конце концов и в каком порядке.

Формат ввода

На первой строке вводится натуральное число N — количество событий.

Далее следуют N строк описанного в условии формата.

Гарантируется, что фамилии людей все разные и что в середину очереди пристраиваются только за кем-то, кто там уже действительно стоит, и отзывают из очереди тоже только тех, кто там уже стоит.

Фамилии могут выглядеть как угодно, в том числе состоять из нескольких слов и включать любые знаки препинания, кроме восклицательного знака.

Формат вывода

Выводится список людей, оставшихся в очереди в конце концов.

Пример

Ввод	Вывод
5 Иванова встала в очередь. Кузнецов встал в очередь. Поливанов встал в очередь. Привет, Кузнецов! Зорина будет за тобой. Иванова, хватит тут стоять, пошли отсюда.	Кузнецов Зорина Поливанов

```

1 s = []
2 for i in range(int(input())):
3     string = input()
4     if 'хватит тут стоять' in string:
5         string = string.split(',')
6         s.pop(s.index(string[0]))
7     elif 'будет за тобой' in string:
8         string = string.split('! ')
9         string_p, string_n = string[1].split(' будет'), string[0].split(', ')
10        name_p, name = string_p[0], string_n[1]
11        s.insert(s.index(name) + 1, name_p)
12    else:
13        s.append(string.split(' s')[0])
14 print('\n'.join(s))

```


Скопировать

Задания к лабораторной работе

1. `s = "У лукоморья 123 дуб зеленый 456"`
 - Определить, встречается ли в строке буква 'я'. Вывести на экран ее позицию (индекс) в строке.
 - Определить, сколько раз в строке встречается буква 'у'.
 - Определить, состоит ли строка только из букв, ЕСЛИ нет, ТО вывести строку в верхнем регистре.
 - Определить длину строки. ЕСЛИ длина строки превышает 4 символа, ТО вывести строку в нижнем регистре.
 - Заменить в строке первый символ на 'О'. Результат вывести на экран

2. Напишите программу, которая запрашивает у пользователя две строки и формирует из этих строк список. Если строки состоят только из чисел, то программа добавляет в середину списка сумму введенных чисел, иначе добавляется строка, образованная из слияния двух введенных ранее строк. Итоговая строка выводится на экран.

3. Напишите программу, которая:
 - принимает на вход строку, состоящую из случайных слов;
 - отбрасывает слова, в которых есть **небуквенные** символы;
 - переставляет слова в порядке возрастания;
 - выводит на экран слова в столбик, без кавычек.

Пример ввода:

```
фрукты образцовый груз@вик цветы невз56годы дер#ево задача виноградник грозовой зима
яб7локо
```

Вывод:

```
зима
цветы
фрукты
задача
грозовой
образцовый
виноградник
```

4. Напишите программу, которая получает на вход строку, и формирует из нее **2** списка:

- в первый список входят слова, начинающиеся с **гласных** букв;
- второй список состоит из слов, начинающихся с **согласных**.

Выведите результат в две строки без кавычек и скобок.

Пример ввода:

```
кот ёж море ель береза апельсин бирюза имитация
```

Вывод:

```
ёж ель апельсин имитация  
кот море береза бирюза
```